

1 Word Embedding

The word embedding stage translates raw textual tokens into dense semantic vectors, establishing a base numeric representation of linguistic meaning.

1.1 Process Steps

1. **Token Representation:** Map each word in the vocabulary $\mathcal{V}$ to a unique binary one-hot encoded vector $\mathbf{x} \in \{0, 1\}^V$, where $V = |\mathcal{V}|$ is the vocabulary size.
2. **Dense Projection:** Project the high-dimensional sparse one-hot vector $\mathbf{x}$ using a learnable linear Embedding Matrix $W_E \in \mathbb{R}^{V \times d_{\text{model}}}$:

$$E_{\text{word}} = \mathbf{x} \cdot W_E \quad (1)$$

where d_{model} is the hyperparameter specifying the dense embedding dimensionality of the model.

2 Positional Encoding (PE)

Because the self-attention mechanism processes all tokens concurrently, it is permutation-invariant and lacks an inherent sense of sequential order. Positional encodings inject absolute and relative temporal coordinates directly into the static word embeddings.

2.1 Coordinate Formulations

For each sequence token position pos and dimension index $j \in \{0, 1, \dots, d_{\text{model}} - 1\}$ (using $j = 2i$ for even dimensions and $j = 2i + 1$ for odd dimensions):

- **Even Dimensions** ($j = 2i$):

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

- **Odd Dimensions** ($j = 2i + 1$):

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3)$$

2.2 Semantic Injection

The positional coordinate matrix PE is added element-wise directly to the static word embedding matrix X_{word} to produce the context-ready sequence matrix X :

$$X = X_{\text{word}} + PE \quad (4)$$

3 Encoder Output (Self-Attention)

The encoder constructs context-rich, multi-directional representations of the entire source sentence, capturing the relationships between all words regardless of distance.

3.1 Projections

We project the positional-embedded input sequence matrix $X \in \mathbb{R}^{L \times d_{\text{model}}}$ using three learnable projection weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ to calculate the Query (Q), Key (K), and Value (V) matrices:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V \quad (5)$$

3.2 Self-Attention Calculation

Using the scaled dot-product attention formulation, we compute similarity scores, scale them by the key dimension d_k (typically $d_k = d_{\text{model}}$), normalize them using a softmax function, and weigh the values:

$$\text{Attn.Out} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (6)$$

3.3 Residual Connection

To facilitate deep gradient flow, we add the original representation X back to the self-attention output:

$$\text{enc} = X + \text{Attn.Out} \quad (7)$$

4 Decoder Output (Masked Self-Attention)

The decoder processes the target sequences generated autoregressively. To prevent “look-ahead cheating” during training, future tokens are structurally blocked.

4.1 Projections

Project the target sequence matrix X_{dec} to obtain decoder queries, keys, and values:

$$Q_{\text{dec}} = X_{\text{dec}} \cdot W_{Q,\text{dec}}, \quad K_{\text{dec}} = X_{\text{dec}} \cdot W_{K,\text{dec}}, \quad V_{\text{dec}} = X_{\text{dec}} \cdot W_{V,\text{dec}} \quad (8)$$

4.2 Masked Self-Attention

We introduce a lower-triangular mask matrix M where $M_{ij} = 0$ for past/present tokens ($i \geq j$) and $M_{ij} = -\infty$ for future tokens ($i < j$):

$$\text{Masked_Attn.Out} = \text{softmax} \left(\frac{Q_{\text{dec}}K_{\text{dec}}^T}{\sqrt{d_k}} + M \right) V_{\text{dec}} \quad (9)$$

The addition of $-\infty$ ensures that the softmax weights for future positions collapse to exactly zero.

4.3 Residual Connection

$$X_{\text{updated}} = X_{\text{dec}} + \text{Masked_Attn.Out} \quad (10)$$

5 Cross-Attention Output

Cross-attention acts as the sequence-to-sequence bridge, allowing the decoder to query the source sentence context generated by the encoder.

5.1 Projections and Memory Retrieval

- **Cross Queries:** Projected from the decoder’s updated state:

$$Q_{\text{cross}} = X_{\text{updated}} \cdot W_{Q,\text{cross}} \quad (11)$$

- **Cross Keys and Values:** Projected from the static, cached encoder output matrix enc :

$$K_{\text{cross}} = \text{enc} \cdot W_{K,\text{cross}}, \quad V_{\text{cross}} = \text{enc} \cdot W_{V,\text{cross}} \quad (12)$$

5.2 Attention Routing and Final State

We compute cross-attention and apply a final residual sum to yield the prediction-ready representation X_{final} :

$$\text{Cross_Attn_Out} = \text{softmax} \left(\frac{Q_{\text{cross}} K_{\text{cross}}^T}{\sqrt{d_k}} \right) V_{\text{cross}} \quad (13)$$

$$X_{\text{final}} = X_{\text{updated}} + \text{Cross_Attn_Out} \quad (14)$$

This output matrix X_{final} is mapped via a linear projection layer and a final Softmax to predict probability distributions over the target vocabulary.